inmind.academy
by inmind.ai

Agentic LLMs I: Concepts & Patterns

From Multilingual RAG to Task-Executing Agents

SPRING 2026

inmind.academy
by inmind.ai

Recap - Where We Left Off

Before we move into agents, we need to close two important gaps from our RAG discussion: what happens when your users speak different languages, and what happens when your documents contain images, tables, and scanned content? We will address both of these first, then spend the rest of the session on making LLMs do things, not just chat.

inmind.academy
by inmind.ai

Today's Roadmap

- **Part 1:** Multilingual RAG: Handling Multiple Languages
- **Part 2:** Multimodal Documents in RAG: Images, Tables, Scanned PDFs
- **Part 3:** Why Agents: From Q&A to Task Execution
- **Part 4:** Reasoning Patterns: ReAct, ReWoo, Plan-and-Execute
- **Part 5:** Tool Calling / Function Calling
- **Part 6:** Designing Tasks as Steps
- **Part 7:** Intro to LangGraph for Orchestration
- **Part 8:** Lab: Build a Simple Agent

inmind.academy
by inmind.ai

Part 1: Multilingual RAG

What happens when your users speak Arabic, French, and English, but your documents are in a mix of all three?

inmind.academy
by inmind.ai

The Multilingual Challenge

Building a customer support chatbot for a company that operates in Lebanon, France, and the US. Your knowledge base contains documents in Arabic, French, and English. Your users might ask questions in any of these languages. Sometimes they even mix languages in a single.

The challenge: if a user asks a question in Arabic, but the answer is in an English document, will your RAG pipeline find it? With a monolingual embedding model trained only on English, the answer is no. The Arabic query vector and the English document vector will be in completely different regions of the embedding space, even if they mean the same thing.

This is the multilingual retrieval problem, and there are three main strategies to solve it.

inmind.academy
by inmind.ai

Strategy 1: Use Multilingual Embedding Models

- **The idea:** Use an embedding model trained on many languages simultaneously. These models map semantically similar text to nearby vectors regardless of language. A question in Arabic and its answer in English will have similar vectors.
- **How it works:** Multilingual models are trained on parallel corpora (same text in multiple languages) so they learn a shared embedding space.
- **Example models:** multilingual-e5-large (HuggingFace, 560M params, 100+ languages), nomic-embed-text (supports multilingual via Ollama), text-embedding-3-small/large from OpenAI (100+ languages), Cohere embed-multilingual-v3.0.
- **Pros:** Simplest approach. No translation needed. One index for all languages. Works for code-switching.
- **Cons:** Cross-lingual retrieval quality is usually 10-20% lower than same-language retrieval. Some language pairs work better than others (e.g., French-English better than Arabic-English).

Strategy 2: Translate the Query at Search Time

- **The idea:** When a user asks a question in Arabic, translate it to English before embedding and searching. The retrieval happens in the document language, then the answer is translated back.
- **Flow:** User query (Arabic) -> LLM translates to English -> Embed English query -> Search English vector DB -> Retrieve English chunks -> Generate answer -> Optionally translate answer back.
- **Pros:** You can use any monolingual embedding model (often higher quality for that language). Retrieval quality stays high because query and documents are in the same language.
- **Cons:** Adds latency (translation step). Translation errors propagate. Doesn't work well for code-switched queries. Extra API cost.

```
# Query translation with an LLM
user_query_arabic = "shu howe lfar2 bayn AI w LLM?"

translate_msg = [
    SystemMessage(content="Translate the following to English. Return only the translation."),
    HumanMessage(content=user_query_arabic)
]
english_query = llm.invoke(translate_msg).content
# Now embed and search with the English query
q_vec = embed_model.embed_query(english_query)
```

Strategy 3: Translate Documents at Indexing Time

- **The idea:** Translate all documents into a single language (or into all target languages) during the indexing phase. Store multiple versions. At query time, search in the user's language.
- **How it works:** For each document, create translations into your target languages. Embed each version. Store all versions in the vector DB with a language metadata field. At query time, search only the index matching the user's language.
- **Pros:** Best retrieval quality (same-language matching). User always gets results in their own language. No query-time translation latency.
- **Cons:** Most expensive approach. Storage multiplied by number of languages. Translation quality affects everything downstream. Keeping translations in sync when documents update is complex.
- **When to use:** When you have a fixed set of target languages, high-quality retrieval is critical, and you can afford the upfront cost of translating your entire corpus.

inmind.academy
by inmind.ai

Part 2: Multimodal Documents in RAG

Real-world documents contain images, tables, charts, headers, and scanned text. Your RAG pipeline needs to handle all of it.

inmind.academy
by inmind.ai

Strategy 1: Better Text Extraction (Tables + Layout)

- **The idea:** Use specialized PDF parsing libraries that understand document layout, extract tables as structured data, and preserve reading order.
- **PyMuPDF:** Fast, extracts text with position info, can extract images. Good for well-structured PDFs.
- **Unstructured.io:** Open-source library built for document parsing. Handles PDFs, Word docs, HTML, images. Classifies elements as titles, text, tables, images. LangChain integration available.
- **LlamaParse:** Cloud-based parsing by LlamaIndex. Very accurate, handles complex layouts. Free tier available.
- **For tables:** Extract tables as markdown or CSV, embed them as separate chunks with metadata like type=table and page=5. This way table data is searchable.
- **Limitation:** Still loses image content entirely. If a diagram contains the answer, this approach will miss it.

Strategy 2: OCR for Scanned Documents

- **The problem:** Scanned PDFs are images, not text. PyPDFLoader returns empty strings for scanned pages.
- **OCR (Optical Character Recognition):** Convert images of text into actual text.
- **Tesseract OCR:** Free, open-source, supports 100+ languages including Arabic. pip install pytesseract. Accuracy varies with scan quality.
- **EasyOCR:** Python library, supports 80+ languages, good for mixed-language documents. Often better than Tesseract for non-Latin scripts.
- **Azure Document Intelligence / Google Document AI:** Cloud APIs with high accuracy, table extraction, and layout understanding. Best quality but costs money.
- **Pipeline:** PDF page -> convert to image (pdf2image) -> run OCR -> get text -> chunk and embed as normal.
- **Quality note:** OCR accuracy depends heavily on scan quality. Low-resolution scans, handwriting, or unusual fonts will produce errors. Always review OCR output quality.

Strategy 3: Vision LLMs for Document Understanding

- The idea: Instead of extracting text from images, send the entire page image to a Vision-Language Model and ask it to describe the content. The VLM can read text, interpret tables, describe charts, and understand diagrams.
- How it works: Convert each PDF page to an image. Send the image to a VLM (GPT-4o, Claude, LLaVA, Gemini). Ask it to describe all content. Chunk and embed the description.
- Pros: Handles everything: text, tables, images, charts, diagrams, handwriting. No separate OCR needed.
- Cons: Expensive (VLM API calls per page). Slow (seconds per page). VLM descriptions may miss details. Not for bulk processing.

```
# Using a VLM to process a PDF page
import base64
from langchain_ollama import ChatOllama
from langchain_core.messages import HumanMessage

with open("page_1.png", "rb") as f:
    img_b64 = base64.b64encode(f.read()).decode()

vlm = ChatOllama(model="llava")
response = vlm.invoke([HumanMessage(content=[
    {"type": "text", "text": "Describe ALL content on this page in detail."},
    {"type": "image_url", "image_url": {"url": f"{img_b64}"}}
])])
```

Strategy 4: Multimodal Embeddings (Advanced)

- **The idea:** Use embedding models that can embed both text and images into the same vector space. Then you can search for images using text queries and vice versa.
- **How it works:** Embed document images directly (not their text descriptions). When a user asks a text question, embed the question. Vector search finds relevant images/pages even without text extraction.
- **Models:** CLIP (OpenAI, open-source), Jina CLIP v2, Nomic Embed Vision, Google Multimodal Embeddings.
- **Pros:** No text extraction or OCR needed. Images are searchable by meaning. A query like "show me the revenue chart" can find the actual chart image.
- **Cons:** Text retrieval quality is lower than text-only embeddings. Works best for image-heavy documents. These models are still maturing for document understanding.
- **When to use:** Image-heavy knowledge bases (product catalogs, design docs, medical imaging). Not recommended as the sole strategy for text-heavy documents.

inmind.academy
by inmind.ai

Multimodal RAG: Choosing a Strategy

Strategy	Text	Tables	Images	Cost	Speed	Best For
Enhanced extraction	Yes	Yes (markdown)	No	Free	Fast	Well-structured PDFs
OCR	Scanned text	Limited	No	Free/Low	Medium	Scanned documents
Vision LLMs	Yes	Yes	Yes	High	Slow	High-value mixed docs
Multimodal embeddings	Via image	Via image	Yes	Medium	Medium	Image-heavy collections

inmind.academy
by inmind.ai

Part 3: Why Agents?

From Q&A to task execution. Making LLMs do things, not just chat.

inmind.academy
by inmind.ai

From Q&A to Task Execution

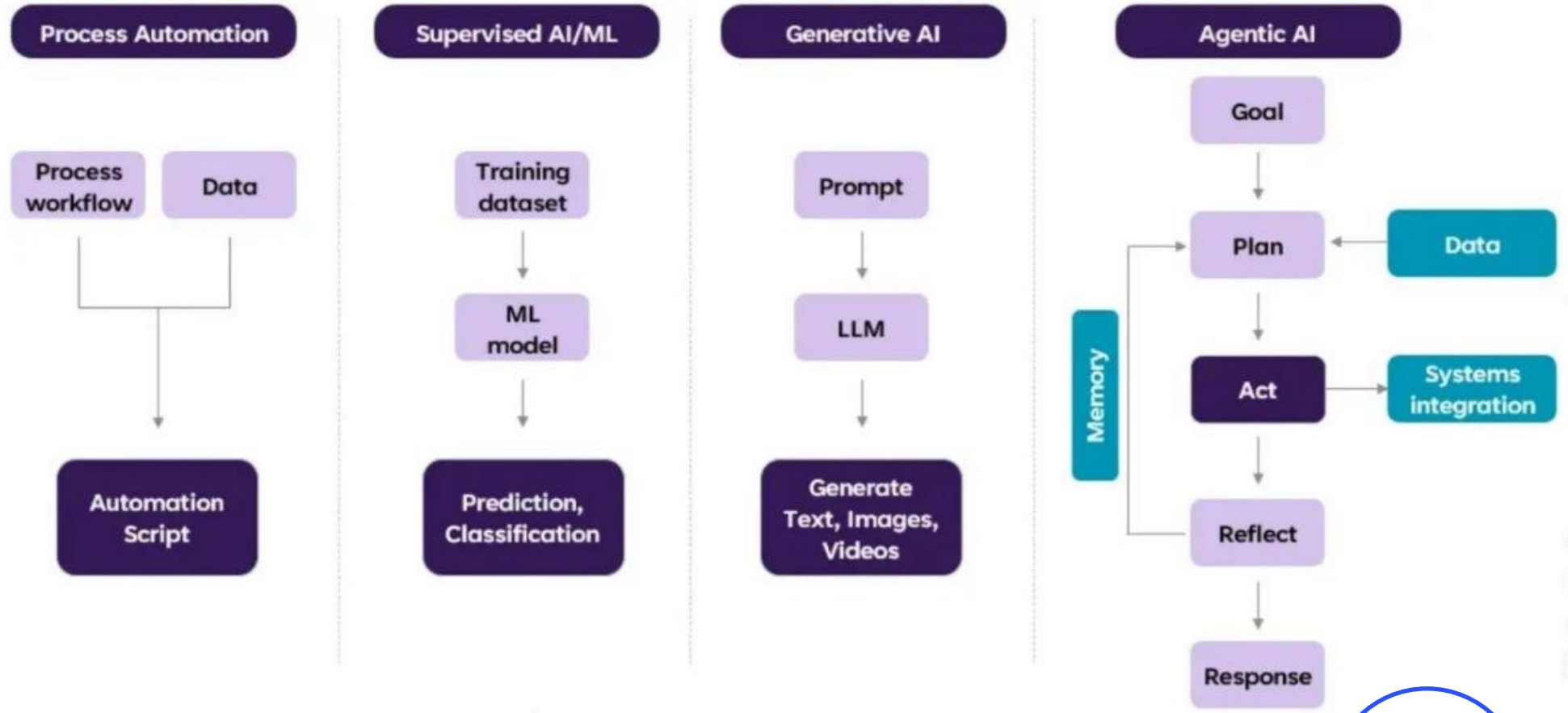
So far, every system we have built follows the same pattern: the user asks a question, and the system answers it. Even with RAG, the flow is linear: question -> retrieve -> generate -> done.

But think about what a human assistant would do if you asked: "Find the cheapest flight from Beirut to Paris next week, check my calendar for conflicts, and book the best option." A human would: search for flights, compare prices, open your calendar, check for conflicts, make a decision, and then book. That is multiple steps, multiple tools, and decision-making along the way.

An LLM agent is an LLM that can do this. Instead of just generating text, it can reason about what steps to take, use tools (search, calculators, APIs, databases, your RAG pipeline), observe the results, and decide what to do next. The LLM becomes the "brain" that orchestrates a workflow.

inmind.academy
by inmind.ai

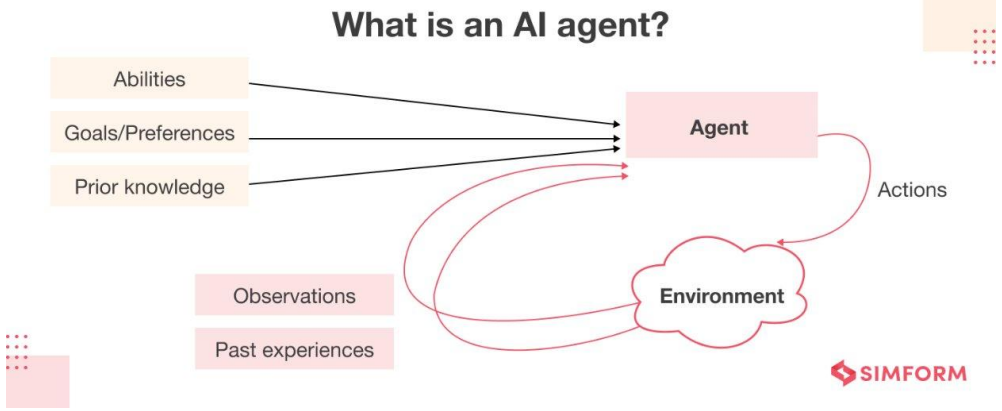
Agentic AI Evolution



inmind.academy
by inmind.ai

What is an LLM Agent?

- **Definition:** An LLM agent is a system where an LLM acts as the reasoning engine that decides which actions to take, executes them via tools, observes the results, and iterates until the task is complete.
- **LLM (the brain):** Reasons about the task, decides next steps, interprets results.
- **Tools:** External functions the agent can call. Examples: web search, calculator, database query, file operations, API calls, your RAG pipeline.
- **Memory:** The conversation history and any intermediate results. The agent needs to remember what it has done so far.
- **Orchestration loop:** The logic that connects the LLM to tools and manages the flow. Think -> Act -> Observe -> Think again.
- **Key insight:** The LLM does not just generate text. It generates structured actions (tool calls), receives real-world results, and adapts its plan based on what it learns. The LLM is in the loop, making decisions at every step.



inmind.academy
by inmind.ai

The Agent Loop

Every agent follows the same fundamental loop:

1. Observe: The agent receives the user's task and any previous observations.

2. Think: The LLM reasons about what to do next.

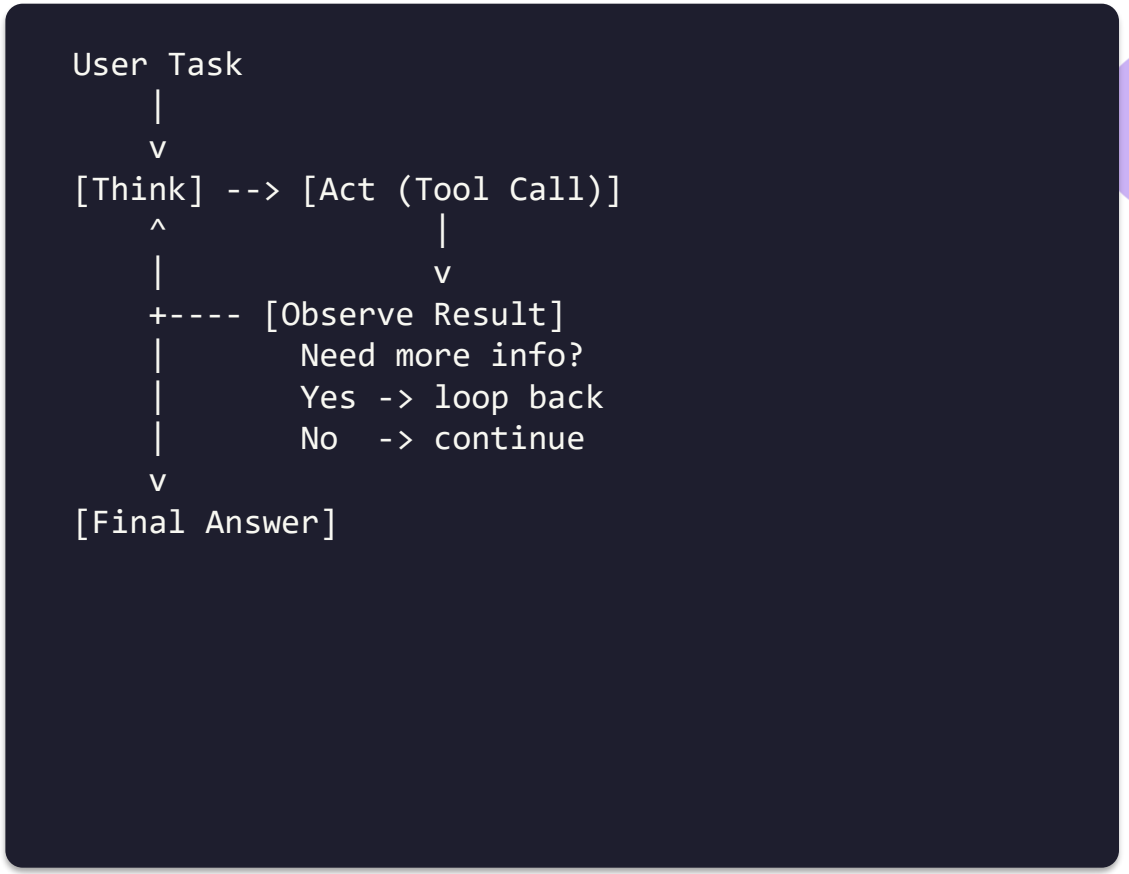
3. Act: The agent calls a tool with specific parameters.

4. Observe: The tool returns results. The agent adds these to its context.

5. Repeat: Does it have enough info? If not, go back to step 2.

6. Respond: When the agent has enough information, it generates the final answer.

This loop is the foundation of all agent architectures.



inmind.academy
by inmind.ai

Part 4: Reasoning Patterns

Different ways to structure how an agent thinks and acts. ReAct, ReWoo, Plan-and-Execute, and when to use each.

inmind.academy
by inmind.ai

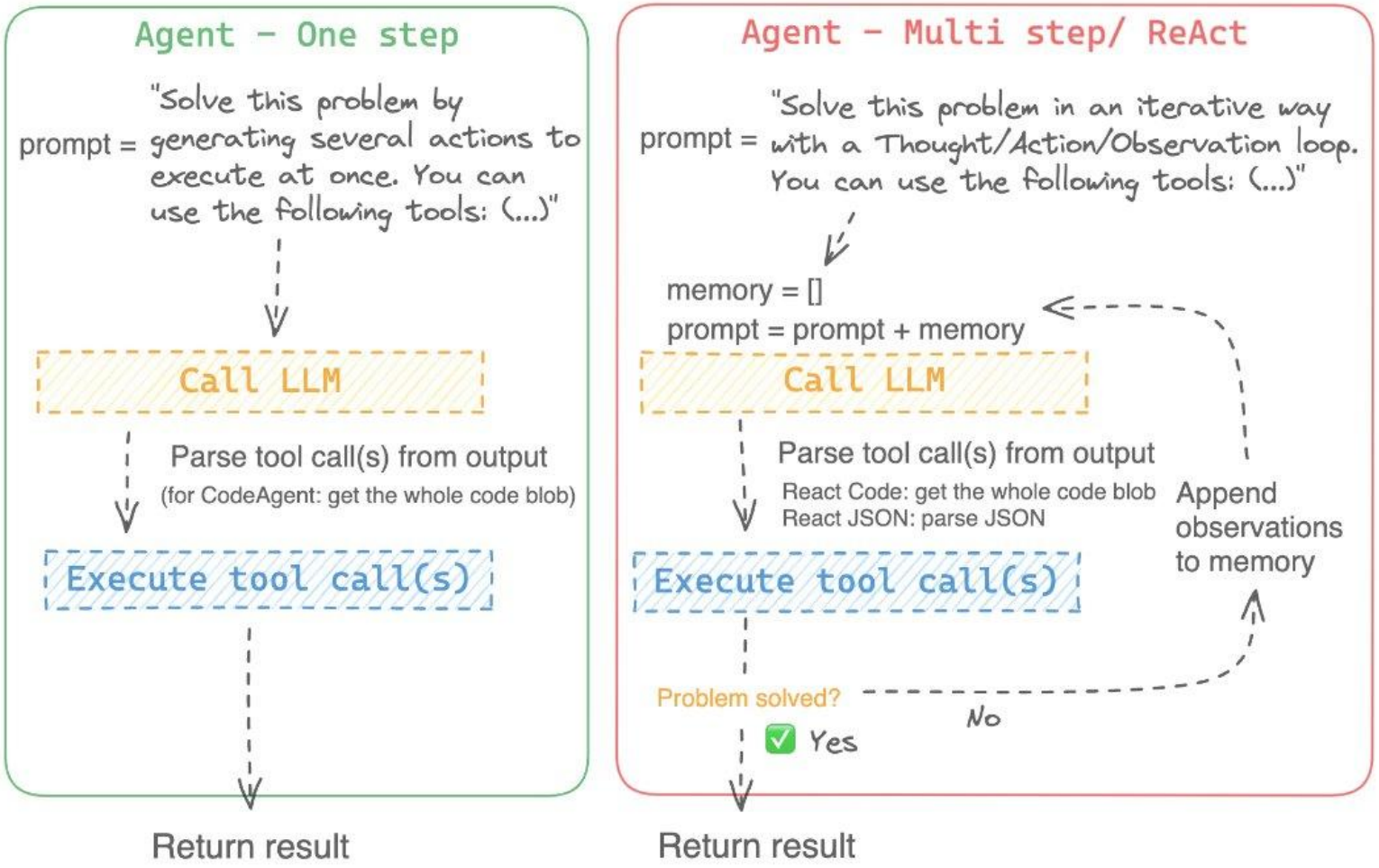
ReAct: Reason + Act (Most Common Pattern)

ReAct (Yao et al., 2022) is the most widely used agent reasoning pattern. The LLM alternates between reasoning ("Thought") and acting ("Action") in an interleaved fashion. After each action, it observes the result and reasons again.

```
Thought: I need to find the population of Lebanon.
Action: search("population of Lebanon 2024")
Observation: Lebanon has an estimated population of 5.3 million.
Thought: Now I need the population of Jordan for comparison.
Action: search("population of Jordan 2024")
Observation: Jordan has an estimated population of 11.5 million.
Thought: I now have both numbers.
Final Answer: Jordan (11.5M) is roughly 2.2x the population of Lebanon (5.3M).
```

- **Why it works:** The explicit Thought step forces reasoning before acting. The interleaved nature means the agent adapts based on each tool result.
- **Limitation:** Each step depends on the previous one, so it is sequential. Slow for tasks with many independent lookups.

ReAct: Reason + Act (Most Common Pattern)



ReWoo: Reasoning Without Observation

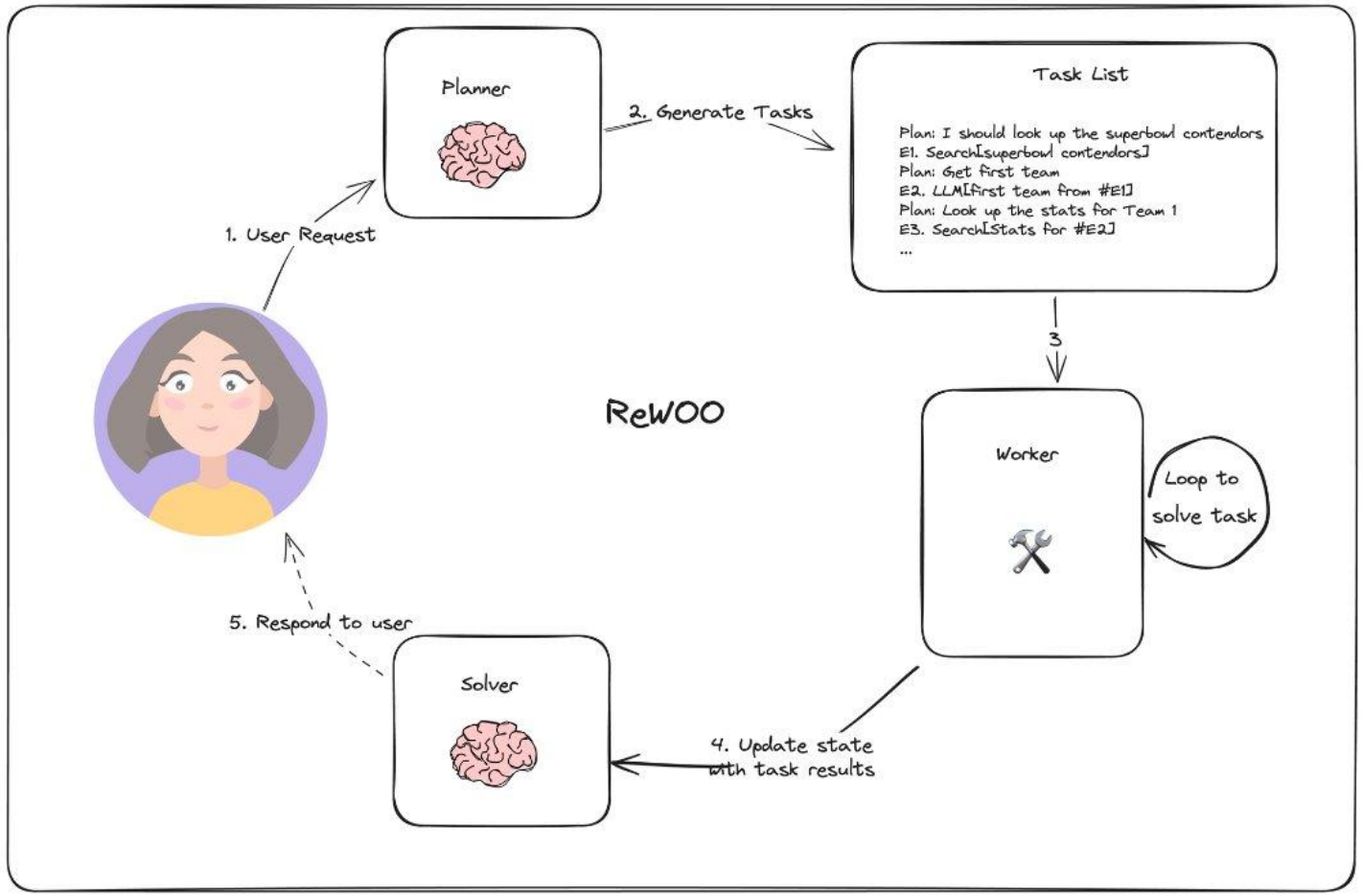
ReWoo (Xu et al., 2023) takes a different approach. Instead of interleaving thought and action, the LLM plans ALL the steps upfront, then executes them all, then synthesizes the final answer.

```
Plan:
  #E1 = search("population of Lebanon 2024")
  #E2 = search("population of Jordan 2024")
  #E3 = calculator("#E1 / #E2")

Execute: (run all three tool calls, possibly in parallel)

Synthesize: Based on the results...
```

- **Pros:** Faster for tasks with independent sub-steps (parallel execution). Fewer LLM calls (one for planning, one for synthesis vs. many for ReAct). More predictable execution.
- **Cons:** Cannot adapt mid-execution. If the first search returns something unexpected, the remaining plan might be wrong. Requires the task to be fully plannable upfront.
- **When to use:** Well-defined tasks with independent sub-steps. Data gathering tasks. Batch processing. Not for tasks that require adaptive reasoning.

inmind.academy
by inmind.ai

ReWoo: Reasoning Without Observation



inmind.academy
by inmind.ai

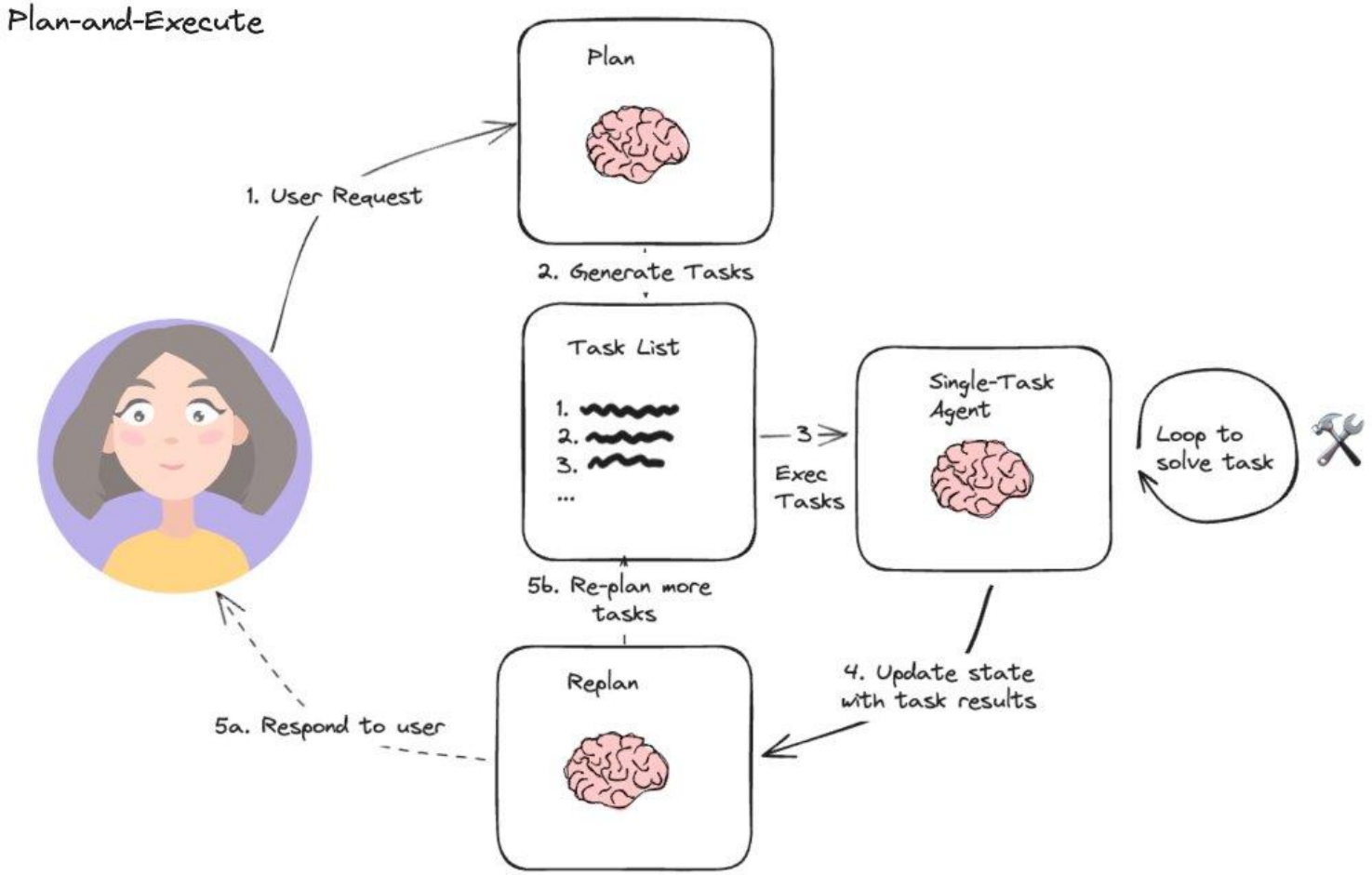
Plan-and-Execute Pattern

Plan-and-Execute is a hybrid. The LLM creates a plan (like ReWoo), then executes steps one at a time (like ReAct), but after each step, it can revise the remaining plan based on what it learned.

```
Plan: 1. Search Lebanon GDP  2. Search population  3. Calculate per capita  4. Compare

Execute Step 1: search("Lebanon GDP 2024")
Observation: GDP is approximately $22 billion.
Re-evaluate plan: Plan still valid, proceed to step 2.

Execute Step 2: search("Lebanon population 2024")
Observation: Population is 5.3 million.
Re-evaluate plan: Plan still valid, proceed to step 3.

Execute Step 3: calculator("22000000000 / 5300000")
Observation: $4,150.94 per capita.
```

- **Pros:** Structured like ReWoo (clear plan), adaptive like ReAct (can revise). Good for complex multi-step tasks.
- **Cons:** More LLM calls than ReWoo (re-evaluation at each step). Plan revision adds complexity.

Plan-and-Execute Pattern



inmind.academy
by inmind.ai

Reasoning Patterns - Comparison

Pattern	Planning	Execution	Adaptability	LLM Calls	Speed	Best For
ReAct	Step-by-step	Sequential	High (adapts each step)	Many (2/step)	Slow	General purpose, uncertain tasks
ReWoo	All upfront	Parallel possible	None (fixed plan)	Few (2 total)	Fast	Well-defined, independent steps
Plan-and-Execute	Upfront + revisions	Sequential	Medium (can revise)	Medium	Medium	Complex multi-step with structure

Practical advice: Start with ReAct. It is the most flexible, the most widely supported in frameworks, and handles the widest range of tasks. Move to Plan-and-Execute when you need more structure. Use ReWoo for batch/parallel workloads.

inmind.academy
by inmind.ai

Part 5: Tool Calling / Function Calling

The mechanism that makes agents possible. How LLMs call external functions with structured parameters.

inmind.academy
by inmind.ai

What is Tool Calling / Function Calling?

Tool calling (also called function calling) is the mechanism that lets an LLM request the execution of an external function with specific arguments. Instead of generating free text, the LLM outputs a structured JSON object specifying which function to call and with what parameters.

This is NOT the LLM executing code. The LLM generates the intent ("I want to call the search function with query 'Lebanon GDP'"), and your application code executes it. The LLM is the decision-maker, your code is the executor.

Tool calling was popularized by OpenAI in June 2023 and is now supported by almost all major LLM providers: OpenAI, Anthropic, Google, Mistral, and even local models via Ollama (with supported models like llama3, mistral, qwen2).

Before tool calling existed, people used prompt engineering to get LLMs to output JSON that looked like function calls. This was unreliable. Modern tool calling is built into the model's training and API, making it much more reliable.

inmind.academy
by inmind.ai

How Tool Calling Works (API Level)

- **Step 1 - Define tools:** Tell the LLM what tools are available, including name, description, and parameter schema (JSON Schema format).
- **Step 2 - User sends a message:** "What's the weather in Beirut?"
- **Step 3 - LLM decides to call a tool:** Instead of generating text, the LLM outputs a tool call: {"name": "get_weather", "arguments": {"city": "Beirut"}}.
- **Step 4 - Your code executes the function:** You receive the tool call, run the actual function, get the result.
- **Step 5 - You send the result back:** Pass the function result back to the LLM as a tool message.
- **Step 6 - LLM generates the final answer:** Using the tool result, the LLM writes a natural language response.
- **Key point:** The LLM can also decide NOT to call a tool and just answer directly. And it can call multiple tools in sequence or even in parallel (if the API supports it).

inmind.academy
by inmind.ai

Defining Tools for an LLM

A tool definition has three parts: name, description, and parameters. The description is critical because the LLM uses it to decide when to call the tool.

```
tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get current weather for a city. Use when user asks about weather.",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {"type": "string", "description": "City name, e.g. 'Beirut'"},
                "units": {"type": "string", "enum": ["celsius", "fahrenheit"]}
            },
            "required": ["city"]
        }
    }
}]
```

- Clear descriptions help the LLM know WHEN to use the tool. Include examples in descriptions. Keep tools to 5-15 (too many confuse the model).

inmind.academy
by inmind.ai

Tool Calling - Complete Code Example (OpenAI)

```
from openai import OpenAI
import json

client = OpenAI()

def search_knowledge_base(query):
    return "Returns are accepted within 30 days with receipt."

tools = [{"type": "function", "function": {
    "name": "search_knowledge_base",
    "description": "Search company docs for policies, products, procedures.",
    "parameters": {"type": "object",
        "properties": {"query": {"type": "string"}}, "required": ["query"]}
}}]

# Step 1: Send message with tools
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "What is the return policy?"}],
    tools=tools)

# Step 2: Check for tool call
msg = response.choices[0].message
if msg.tool_calls:
    tc = msg.tool_calls[0]
    result = search_knowledge_base(**json.loads(tc.function.arguments))

    # Step 3: Send result back
    final = client.chat.completions.create(model="gpt-4o-mini",
        messages=[{"role": "user", "content": "What is the return policy?"},
                  msg, {"role": "tool", "tool_call_id": tc.id, "content": result}],
        tools=tools)
    print(final.choices[0].message.content)
```

inmind.academy
by inmind.ai

Tool Calling with LangChain (Simplified)

```python
from langchain_ollama import ChatOllama
from langchain_core.tools import tool

# Define tools using the @tool decorator
@tool
def search_knowledge_base(query: str) -> str:
    """Search company docs for product info, policies, or procedures."""
    # Your RAG pipeline here!
    return "Returns are accepted within 30 days with receipt."

@tool
def calculator(expression: str) -> str:
    """Calculate a mathematical expression. Input: valid math expression."""
    return str(eval(expression))

# Bind tools to the model
llm = ChatOllama(model="llama3", temperature=0)
llm_with_tools = llm.bind_tools([search_knowledge_base, calculator])

# Invoke - the model decides whether to call a tool
response = llm_with_tools.invoke("What is 25 * 47?")

# Check for tool calls
if response.tool_calls:
    for tc in response.tool_calls:
        print(f"Tool: {tc['name']}, Args: {tc['args']}")
        if tc['name'] == 'calculator':
            result = calculator.invoke(tc['args'])
            print(f"Result: {result}")
```

inmind.academy
by inmind.ai

Mini-Exercise: Define and Call a Tool

```
from langchain_ollama import ChatOllama
from langchain_core.tools import tool

@tool
def get_country_info(country: str) -> str:
    """Look up basic info about a country: capital, population, region."""
    data = {
        "Lebanon": "Capital: Beirut, Population: 5.3M, Region: Middle East",
        "France": "Capital: Paris, Population: 67M, Region: Western Europe",
        "Japan": "Capital: Tokyo, Population: 125M, Region: East Asia",
    }
    return data.get(country, f"No info found for {country}")

llm = ChatOllama(model="llama3", temperature=0)
llm_with_tools = llm.bind_tools([get_country_info])

for q in ["Capital of Lebanon?", "What is 2 + 2?", "Tell me about Japan."]:
    print(f"\nQ: {q}")
    resp = llm_with_tools.invoke(q)
    if resp.tool_calls:
        for tc in resp.tool_calls:
            print(f"  Tool: {tc['name']}({tc['args']})")
            print(f"  Result: {get_country_info.invoke(tc['args'])}")
    else:
        print(f"  Direct: {resp.content}")
```

inmind.academy
by inmind.ai

Part 6: Designing Tasks as Steps

Complex tasks need to be broken down into manageable steps. This is where agent design becomes software design.

inmind.academy
by inmind.ai

Designing Tasks as Steps

The hardest part of building an agent is not the code, it is the design. You need to break down a complex task into steps that an LLM can handle one at a time.

Design principles:

- Each step should have a clear, single objective
- Each step should produce an observable output that can be verified
- Steps should be ordered logically (dependencies first)
- The LLM should have enough context at each step to make a good decision
- Fail gracefully: what happens if a tool call returns an error?

Example: "Summarize all PDFs in this folder and email them to my team."

- **Step 1:** List all PDF files in the folder (tool: file_system)
- **Step 2:** For each PDF, extract text (tool: pdf_reader)
- **Step 3:** For each extracted text, generate a summary (tool: llm)
- **Step 4:** Combine all summaries into one document (tool: llm)
- **Step 5:** Send email with the combined summary (tool: email_sender)

inmind.academy
by inmind.ai

Real-World Task Decomposition

Task: "Research our competitor's latest product and draft a comparison report."

```
Step 1: [Search] Look up competitor's latest product announcements
        Tool: web_search("CompanyX latest product 2025")
        Output: List of product details

Step 2: [RAG] Retrieve our own product specs from internal docs
        Tool: rag_query("our product specifications features")
        Output: Our product details

Step 3: [Analyze] Compare features side by side
        Tool: LLM reasoning (no external tool needed)
        Output: Comparison analysis

Step 4: [Generate] Draft the comparison report
        Tool: LLM generation
        Output: Formatted report

Step 5: [Save] Save report to file
        Tool: file_write("comparison_report.md", report_text)
```

Notice: Step 2 uses your RAG pipeline as a tool. This is how RAG and agents connect. Your RAG system from Session 8 becomes one of many tools an agent can use.

inmind.academy
by inmind.ai

Part 7: LangGraph for Orchestration

A framework for building agents as graphs of nodes and edges. Structured, debuggable, and production-ready.

inmind.academy
by inmind.ai

Why LangGraph?

You could build an agent with a simple while loop (as we showed in the ReAct example). For simple agents, that works fine. But real-world agents need: conditional branching (if tool returns error, retry or try different approach), parallel execution (call multiple tools at once), human-in-the-loop checkpoints (pause and ask the user before proceeding), state management (track what the agent has done, what it knows), persistence (resume an interrupted agent), and streaming (show the user what the agent is doing).

LangGraph (by LangChain) provides all of this by modeling the agent as a directed graph. Nodes are functions (LLM calls, tool calls, custom logic). Edges are transitions between nodes (conditional or unconditional). State is a shared object that flows through the graph.

LangGraph is NOT LangChain's old AgentExecutor (which is deprecated). It is a separate, lower-level framework that gives you full control.

inmind.academy
by inmind.ai

LangGraph - Core Concepts

- **State:** A TypedDict or Pydantic model that holds all data flowing through the graph. Every node reads from and writes to this state. Example: {"messages": [], "tool_results": [], "final_answer": None}.
- **Nodes:** Functions that take the current state, do something (call an LLM, run a tool, process data), and return updated state. Each node is a step in your agent.
- **Edges:** Connections between nodes. Can be unconditional (always go from A to B) or conditional (if result is X go to B, if Y go to C).
- **Graph:** The full workflow. Build by adding nodes and edges, then compile. The compiled graph is executable.
- **Special nodes:** START (entry point) and END (exit point). Every graph must have a path from START to END.
- **Checkpointing:** LangGraph can save state at each node, enabling human-in-the-loop workflows and recovery from failures.

inmind.academy
by inmind.ai

LangGraph - Simple Chatbot (No Tools)

```python
from langgraph.graph import StateGraph, START, END
from langchain_ollama import ChatOllama
from langchain_core.messages import HumanMessage
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages

# 1. Define the state
class State(TypedDict):
    messages: Annotated[list, add_messages]

# 2. Define the node
llm = ChatOllama(model="llama3", temperature=0)

def chatbot(state: State):
    response = llm.invoke(state["messages"])
    return {"messages": [response]}

# 3. Build the graph
graph = StateGraph(State)
graph.add_node("chatbot", chatbot)
graph.add_edge(START, "chatbot")
graph.add_edge("chatbot", END)

# 4. Compile and run
app = graph.compile()
result = app.invoke({"messages": [HumanMessage(content="What is RAG?")]})
print(result["messages"][-1].content)
```

inmind.academy
by inmind.ai

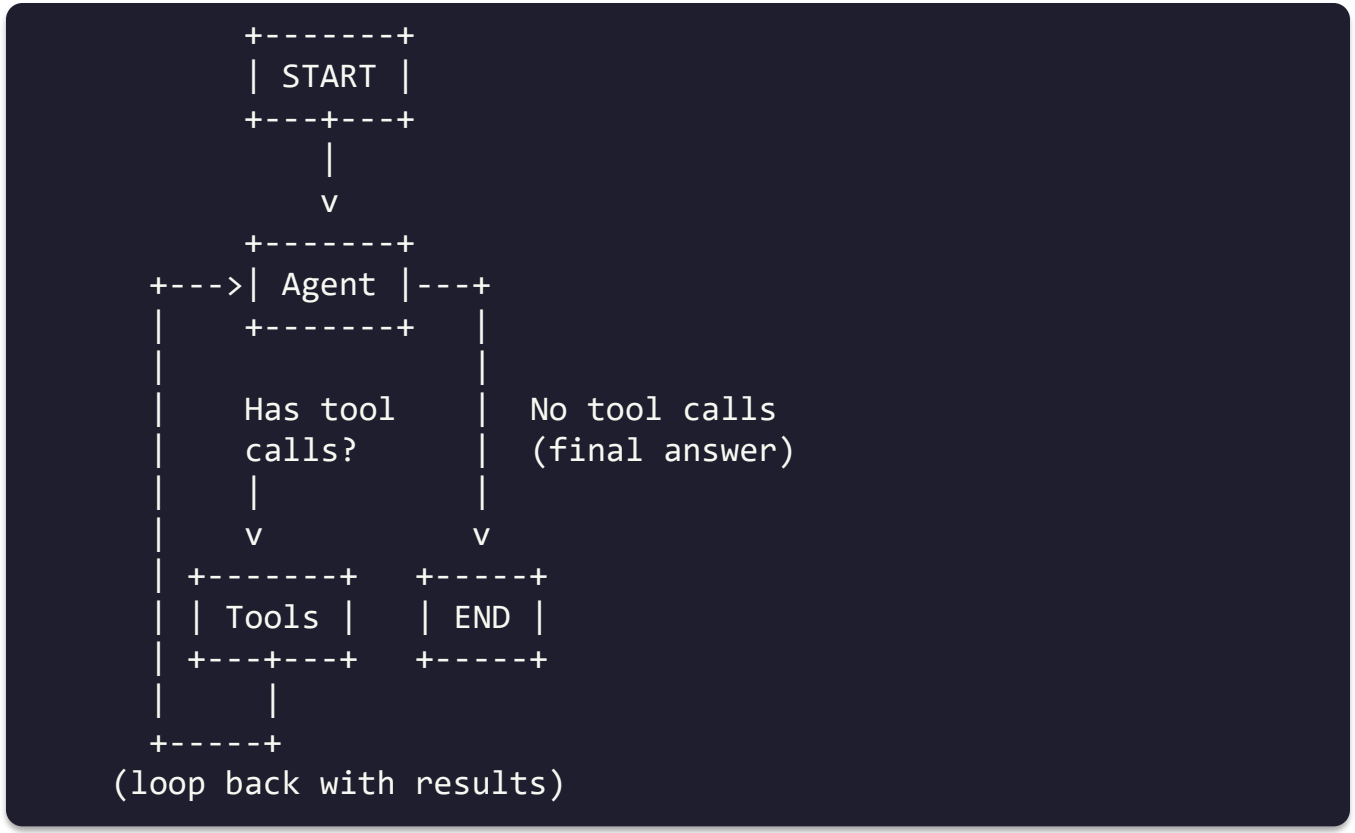
LangGraph - ReAct Agent with Tools

```python
from langgraph.graph import StateGraph, START, END
from langgraph.prebuilt import ToolNode
from langchain_ollama import ChatOllama
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage
from typing import TypedDict, Annotated, Literal
from langgraph.graph.message import add_messages

@tool
def search(query: str) -> str:
    """Search the web for current information."""
    return f"Results for '{query}': Lebanon GDP is approx $22B."

@tool
def calculator(expression: str) -> str:
    """Evaluate a math expression."""
    return str(eval(expression))

tools = [search, calculator]
llm = ChatOllama(model="llama3", temperature=0).bind_tools(tools)

class State(TypedDict):
    messages: Annotated[list, add_messages]

def agent(state): return {"messages": [llm.invoke(state["messages"])]}

def should_continue(state) -> Literal["tools", "end"]:
    if state["messages"][-1].tool_calls: return "tools"
    return "end"
```

```python
graph = StateGraph(State)
graph.add_node("agent", agent)
graph.add_node("tools", ToolNode(tools))
graph.add_edge(START, "agent")
graph.add_conditional_edges("agent", should_continue, {"tools": "tools",
"end": END})
graph.add_edge("tools", "agent")
app = graph.compile()
```

inmind.academy
by inmind.ai

LangGraph - Agent Flow Visualized



This is the ReAct pattern as a graph. The agent node calls the LLM. If the LLM decides to use tools, we go to the tools node (which executes them). Then we loop back to the agent with the results. If the LLM decides it has enough info, we go to END.

inmind.academy
by inmind.ai

Connecting RAG and Agents

Your RAG pipeline from Session 8 becomes a tool that the agent can call. This is one of the most powerful patterns in production LLM applications.

```
@tool
def search_company_docs(query: str) -> str:
    """Search internal company documents for policies, procedures, product info."""
    # This is your RAG pipeline from Session 8!
    q_vec = embed_model.embed_query(query)
    results = collection.query(query_embeddings=[q_vec], n_results=3)
    return "\n".join(results["documents"][0])

@tool
def search_web(query: str) -> str:
    """Search the internet for current, public information."""
    return "Web search results..."

# The agent decides WHICH tool to use based on the question:
# "What is our refund policy?" -> search_company_docs
# "What is the current USD/LBP rate?" -> search_web
# "What is 15% of $200?" -> calculator
tools = [search_company_docs, search_web, calculator]
llm_with_tools = llm.bind_tools(tools)
```

The agent decides WHICH tool to use based on the question. Internal question -> RAG. External question -> web search. Math -> calculator.

inmind.academy
by inmind.ai

THANK YOU

Questions?

ADDRESS

Mkalles, main road, bld. 757, Lebanon

CONTACT

Phone +961 70 478 758
Email inmind.academy@inmind.ai
Website www.inmind.academy

SOCIAL MEDIA